

API Reference

Contents

1 Sandkit API Method Reference

1.1 Engine escape hatch

1.2 Main entry

1.2.1 api.gameConfig

1.2.2 api.action

1.2.3 api.assets

1.2.4 api.authorization

1.2.5 api.building

1.2.6 api.camera

1.2.7 api.scene

1.2.8 api.collector

1.2.9 api.cooldown

1.2.10 api.discoveries

1.2.11 api.effects

1.2.12 api.elements

1.2.13 api.energy

1.2.14 api.excavation

1.2.15 api.events

1.2.16 api.fire

1.2.17 api.grid

1.2.18 api.hooks

1.2.19 api.i18n

1.2.20 api.input

1.2.21 api.items

1.2.22 api.lights.vfx

1.2.23 api.lights.persistent

1.2.24 api.maps

1.2.25 api.mods

1.2.26 api.patterns

1.2.27 api.player

1.2.28 api.player.inventory

1.2.29 api.player.buildings

1.2.30 api.progression

1.2.31 api.processing

- 1.2.32 api.projectiles
- 1.2.33 api.random
- 1.2.34 api.raycast
- 1.2.35 api.reactions
- 1.2.36 api.rendering
- 1.2.37 api.resources
- 1.2.38 api.schedule
- 1.2.39 api.signals.targets
- 1.2.40 api.sound
- 1.2.41 api.sprites
- 1.2.42 api.storage
- 1.2.43 api.storage.local
- 1.2.44 api.settings
- 1.2.45 api.structureBehaviors
- 1.2.46 api.structures.recipes
- 1.2.47 api.structures.processing
- 1.2.48 api.structures
- 1.2.49 api.tech
- 1.2.50 api.time
- 1.2.51 api.tools.grabber
- 1.2.52 api.terrains
- 1.2.53 api.triggers
- 1.2.54 api.ui
- 1.2.55 api.ui.overlays
- 1.2.56 api.ui.navigation
- 1.2.57 api.upgrades
- 1.2.58 api.utils
- 1.2.59 api.world
- 1.2.60 api.world.pickups
- 1.2.61 api.shared.buffers
- 1.2.62 api.workers

1.3 Worker entry

- 1.3.1 api.collector
- 1.3.2 api.effects
- 1.3.3 api.elements
- 1.3.4 api.events
- 1.3.5 api.hooks
- 1.3.6 api.maps
- 1.3.7 api.fire

- 1.3.8 [api.patterns](#)
- 1.3.9 [api.player](#)
- 1.3.10 [api.random](#)
- 1.3.11 [api.structures.processing](#)
- 1.3.12 [api.structures](#)
- 1.3.13 [api.terrains](#)
- 1.3.14 [api.ui](#)
- 1.3.15 [api.utils](#)
- 1.3.16 [api.world](#)
- 1.3.17 [api.main](#)
- 1.3.18 [api.worker](#)
- 1.3.19 [api.shared.buffers](#)

Sandkit API Method Reference

Engine escape hatch

`engine.api` is the internal API.

```
sandkit.engine.api;  
sandkit.engine.state;
```

Main entry

`api` refers to `sandkit.api`.

`api.gameConfig`

```
api.gameConfig.get(key: string): JsonValueV1 | undefined;  
api.gameConfig.getAll(): JsonObjectV1;
```

`api.action`

```
api.action.getActive(): Action;  
api.action.getSelected(): Action;  
api.action.setCustomData(data: any): void;
```

api.assets

```
api.assets.getUrl(relativePath: string): string;  
api.assets.getSelectedProvider(kind: string): AssetProviderV1 | null;  
api.assets.selectProvider(kind: string, providerId: string | null): boolean;
```

api.authorization

```
api.authorization.canBuildAtCell(cellX: number, cellY: number): boolean;  
api.authorization.canGrabAtCell(cellX: number, cellY: number): boolean;  
api.authorization.canUseTool(player: Player, isFlamethrower?: boolean): boolean;  
api.authorization.canUseToolAtCell(cellX: number, cellY: number, isFlamethrower?: boolean): boolean;  
api.authorization.getZoneIdAtCell(cellX: number, cellY: number): number;  
api.authorization.getPlayerZoneId(): number;
```

api.building

```
api.building.getSnappedPositionAtCell(cellX: number, cellY: number): { x: number; y: number; };  
api.building.isBlockedAtCell(cellX: number, cellY: number): boolean;  
api.building.cancelPlacement(): void;  
api.building.selectStructure(structureTypeOrId: string | StructureType): string | StructureType | null;
```

api.camera

```
api.camera.snapToPlayer(): void;  
api.camera.setFocusAtWorld(worldX: number, worldY: number): boolean;  
api.camera.releaseFocus(options?: { durationMs?: number; }): boolean;
```

api.scene

```
api.scene.getActive(): Scene;
```

api.collector

```
api.collector.getValueFromCellId(cellId: number): number;  
api.collector.getValueByType(elementType: number): number;  
api.collector.isCellIdCollectable(cellId: number): boolean;  
api.collector.isCellIdCollectableForSprite(cellId: number): boolean;  
api.collector.notifyPickupAtCell(cellX: number, cellY: number): void;
```

api.cooldown

```
api.cooldown.check(cooldown: Cooldown, overrideTime?: number): boolean;  
api.cooldown.isReady(cooldown: Cooldown, overrideTime?: number): boolean;
```

api.discoveries

```
api.discoveries.addElementByType(elementType: number): void;  
api.discoveries.addTerrainByType(terrainType: number): void;
```

api.effects

```
api.effects.createDistortionWaveAtWorld(worldX: number, worldY: number, options?: { style?: 'implode' | 'explode'; duration?: number; maxRadius?: number; intensity?: number; color?: [number, number, number, number]; }): void;  
api.effects.createEffectAtWorld(effectId: string, worldX: number, worldY: number, options?: any): void;  
api.effects.createLaserAtWorld(startWorldX: number, startWorldY: number, endWorldX: number, endWorldY: number, options?: { width?: number; brightness?: number; color?: number; glow?: boolean; }): any;  
api.effects.createLightAtWorld(worldX: number, worldY: number, options?: TemporaryLightOptions): { index: number | null; };  
api.effects.createParticlesAtWorld(worldX: number, worldY: number, options?: ParticleEffectOptions): void;  
api.effects.removeLightById(lightId: number): void;
```

api.elements

```
api.elements.getRegisteredTypes(): ElementType[];  
api.elements.register(definition: ElementDefinition): { elementType: ElementType; };  
api.elements.updateDefinition(elementTypeOrId: string | ElementType, partial: Partial<ElementDefinition>): void;  
api.elements.addInteractionInfo(elementTypeOrId: string | ElementType, interaction: Interaction): void;  
api.elements.getTypeFromId(elementId: string): ElementType;  
api.elements.getNameByType(elementType: number): string;  
api.elements.getDefinitionByType(elementType: ElementType): ElementDefinition | undefined;  
api.elements.getTypeAtCell(cellX: number, cellY: number): ElementType | null;  
api.elements.getResolvedTypeAtCell(cellX: number, cellY: number): ElementType | null;  
api.elements.getResolvedTypeFromCellId(cellId: number): ElementType | null;  
api.elements.getInfoAtCell(cellX: number, cellY: number): { elementType: ElementType; isParticle: boolean; cellId: number; elementIndex: number; } | null;  
api.elements.getMatterTypeAtCell(cellX: number, cellY: number): MatterType | null;  
api.elements.isTypeAtCell(cellX: number, cellY: number, elementType: ElementType): boolean;
```

```

api.elements.isFreeFallingAtCell(cellX: number, cellY: number): boolean;
api.elements.findFreeCellInStructure(structureCellX: number, structureCellY: number, structureSize: number):
{ x: number; y: number; } | null;
api.elements.createAtCellWhenIdle(cellX: number, cellY: number, elementType: ElementType, options?:
ElementCreateOptions): void;
api.elements.replaceAtCellWhenIdle(cellX: number, cellY: number, elementType: ElementType, options?:
ElementCreateOptions): void;
api.elements.removeAtCellWhenIdle(cellX: number, cellY: number, options?: ElementRemovalOptions): void;
api.elements.teleportBetweenCellsWhenIdle(fromCellX: number, fromCellY: number, toCellX: number, toCellY:
number): void;
api.elements.getVelocityAtCell(cellX: number, cellY: number): { x: number; y: number; } | null;
api.elements.setVelocityAtCellWhenIdle(cellX: number, cellY: number, velocity: { x: number; y: number; }):
void;
api.elements.addParticleVelocityAtCellWhenIdle(cellX: number, cellY: number, velocity: { x: number; y:
number; }, maxSpeed?: number): void;
api.elements.convertToParticleAtCellWhenIdle(cellX: number, cellY: number, velocity: { x: number; y: number;
}): void;
api.elements.convertFromParticleAtCellWhenIdle(cellX: number, cellY: number): void;
api.elements.getDataFieldAtCell(cellX: number, cellY: number, fieldNumber: 1 | 2 | 3 | 4): number | null;
api.elements.setDataFieldAtCellWhenIdle(cellX: number, cellY: number, fieldNumber: 1 | 2 | 3 | 4, value:
number): void;
api.elements.refreshColorAtCellWhenIdle(cellX: number, cellY: number): void;
api.elements.setPhysicsAtCellWhenIdle(cellX: number, cellY: number, physicsState: number): void;
api.elements.setDurationAtCellWhenIdle(cellX: number, cellY: number, duration: number, options?: {
updateMax?: boolean; }): void;

```

api.energy

```

api.energy.registerType(structureId: string, type: 'conductor' | 'storage', options?: any): void;
api.energy.addAtCell(cellX: number, cellY: number, amount: number, options?: any): number;
api.energy.consume(amount: number, options?: { allOrNothing?: boolean; }): number;
api.energy.consumeExcludingNetworkAtCell(cellX: number, cellY: number, amount: number): number;
api.energy.getNetworkAtCell(cellX: number, cellY: number): { x: number; y: number; type: string; }[];
api.energy.getNetworkFreeCapacityAtCell(cellX: number, cellY: number): number;

```

api.excavation

```

api.excavation.registerProfile(id: string, definition: ExcavationProfileDefinitionV1): void;

```

api.events

```

api.events.on<K extends string>(eventId: K, callback: (payload: EventPayload<K>) => void): () => void;
api.events.emit<K extends string>(eventId: K, payload: EventPayload<K>): void;

```

api.fire

```

api.fire.canBurnElementAtCell(cellX: number, cellY: number): boolean;

```

```
api.fire.burnElementAtCellWhenIdle(cellX: number, cellY: number): void;
```

api.grid

```
api.grid.forEachCellInRect(cellX: number, cellY: number, width: number, height: number, callback: (cellX: number, cellY: number) => void): void;  
api.grid.forEachCellInCircle(centerCellX: number, centerCellY: number, radius: number, callback: (cellX: number, cellY: number) => void): void;
```

api.hooks

```
api.hooks.intercept<K extends keyof InterceptHookMap>(hookId: K, callback: (args: InterceptHookMap[K], context: HookContext) => void, options?: HookOptions): () => void;  
api.hooks.modify<K extends keyof ModifierHookMap>(hookId: K, callback: (args: ModifierHookMap[K]) => void, options?: HookOptions): () => void;
```

api.i18n

```
api.i18n.t(key: string, params?: Record<string, string | number>): string;  
api.i18n.register(locale: string, translations: Record<string, string>): void;  
api.i18n.getLocale(): string;  
api.i18n.hasTranslation(key: string, locale?: string): boolean;  
api.i18n.setLocale(locale: string): Promise<void>;  
api.i18n.getLanguages(): { code: string; nativeName: string; englishName: string; enabled: boolean; }[];  
api.i18n.getAvailableLocales(): string[];  
api.i18n.formatNumber(value: number, options?: I18nNumberFormatOptions): string;  
api.i18n.key(...parts: string[]): string;  
api.i18n.getName(definition: { nameKey?: string; name?: string; }): string;  
api.i18n.getDescription(definition: { descriptionKey?: string; description?: string; }): string;  
api.i18n.translatable(key: string, fallback: string): { __translatable: true; key: string; fallback: string; };  
api.i18n.setGlobal(key: string, value: string | (() => string)): void;  
api.i18n.getGlobal(key: string): string | undefined;  
api.i18n.clearGlobal(key: string): void;  
api.i18n.getGlobals(): Record<string, string>;  
api.i18n.formatKeyForDisplay(keyCode: string): string;
```

api.input

```
api.input.registerBinding(bindingId: string, defaultKeys: string[], definition: InputBindingDefinition): string;  
api.input.getMouseCellPosition(): { x: number; y: number; };  
api.input.getBoundKeys(bindingId: string): string[];  
api.input.getDisplayKey(bindingId: string, defaultLabel?: string): string;  
api.input.triggerBinding(bindingId: string): void;  
api.input.pressBinding(bindingId: string): void;  
api.input.releaseBinding(bindingId: string): void;
```

```
api.input.resetMouseState(): void;  
api.input.isCtrlHeld(): boolean;  
api.input.isAltHeld(): boolean;
```

api.items

```
api.items.register(definition: any): void;  
api.items.updateDefinition(itemId: string, partial: Record<string, any>): void;  
api.items.getDefinitionById(itemId: string): any;  
api.items.createFromId(itemId: string): ModItem;  
api.items.getActive(): any;  
api.items.isActiveById(itemId: string | number, itemType?: ItemType): boolean;
```

api.lights.vfx

```
api.lights.vfx.createAtWorld(worldX: number, worldY: number, options?: TemporaryLightOptions): { index:  
number | null; };  
api.lights.vfx.removeById(lightId: number): void;
```

api.lights.persistent

```
api.lights.persistent.createAtWorld(worldX: number, worldY: number, options?: PersistentLightOptions): any;  
api.lights.persistent.removeAtWorld(worldX: number, worldY: number): void;  
api.lights.persistent.fadeAtWorld(worldX: number, worldY: number, durationMs?: number): void;  
api.lights.persistent.markDirty(): void;
```

api.maps

```
api.maps.getActive(): Readonly<ActiveMapV1> | null;  
api.maps.getAvailable(): readonly Readonly<AvailableMapV1>[];  
api.maps.start(mapId: string): boolean;
```

api.mods

```
api.mods.getProviders(kind: string): readonly AssetProviderV1[];
```

api.patterns

```
api.patterns.createCircle(size: number): number[][];  
api.patterns.excavateAtCell(cellX: number, cellY: number, pattern: number[][], outVelocity: { x: number; y: number; }, power: number, options?: PatternExcavateOptions): void;
```

api.player

```
api.player.getWorldPosition(): { x: number; y: number; };  
api.player.setWorldPosition(worldX: number, worldY: number): void;  
api.player.setVelocity(velocityX: number, velocityY: number): void;  
api.player.setMovementSpeedMultiplier(multiplier: number): void;  
api.player.setMovementMode(mode: 'normal' | 'hover'): boolean;  
api.player.isOnGround(): boolean;  
api.player.teleportToGround(): void;  
api.player.isCollidingWithCell(cellX: number, cellY: number): boolean;  
api.player.isWithinRadiusOfCell(cellX: number, cellY: number, radius: number): boolean;  
api.player.isWorldPositionClear(worldX: number, worldY: number): boolean;
```

api.player.inventory

```
api.player.inventory.addFromId(itemId: string): void;
```

api.player.buildings

```
api.player.buildings.unlockByType(structureId: string): void;
```

api.progression

```
api.progression.complete(request: ProgressionCompletionRequestV1): boolean;
```

api.processing

```
api.processing.registerGrower(definition: PlanterBoxRecipeDefinitionV1): void;  
api.processing.registerShaker(definition: ShakerRecipeDefinitionV1): void;  
api.processing.registerKineticPress(definition: KineticPressRecipeDefinitionV1): void;
```

api.projectiles

```
api.projectiles.register(definition: any): void;  
api.projectiles.getDefinitionById(projectileId: string): any;  
api.projectiles.createBlueprintFromId(projectileId: string): ProjectileBlueprint;  
api.projectiles.getAll(): Projectile[];  
api.projectiles.getById(projectileId: number): Projectile | undefined;  
api.projectiles.remove(projectile: Projectile): void;  
api.projectiles.spawnAtWorld(worldX: number, worldY: number, angle: number, blueprint: ProjectileBlueprint):  
Projectile;
```

api.random

```
api.random.int(min: number, max: number): number;  
api.random.float(min: number, max: number): number;
```

api.raycast

```
api.raycast.castFromWorld(startWorldX: number, startWorldY: number, angle: number, maxDistance: number): {  
x: number; y: number; distance: number; } | null;
```

api.reactions

```
api.reactions.registerContact(definition: ContactRecipeDefinitionV1): void;
```

api.rendering

```
api.rendering.getDrawPositionAtCell(cellX: number, cellY: number): { x: number; y: number; };  
api.rendering.getGridMetrics(): { cellSize: number; snapGridCellSize: number; };  
api.rendering.getOverlayViewportSize(): { width: number; height: number; };  
api.rendering.withOverlayContext<T>(callback: (context: CanvasRenderingContext2D) => T): T;
```

api.resources

```
api.resources.collectFluxiteAtCell(cellX: number, cellY: number): void;  
api.resources.updateEnergy(amount: number, options?: { deferUi?: boolean; }): void;
```

api.schedule

```
api.schedule.nextTick(callback: () => void): void;
```

api.signals.targets

```
api.signals.targets.register(structureTypeOrId: string | StructureType, apply: (structure: Structure, payload: SignalTargetPayloadV1) => void): void;
```

api.sound

```
api.sound.play(soundId: string, options?: any): SoundHandle;  
api.sound.playActive(soundId: string, options?: any): SoundHandle;  
api.sound.playLayers(layers: SoundLayer[], options?: { position?: { x: number; y: number; }; volume?: number; rateLimitKey?: string; rateLimitMs?: number; }): SoundHandle[];  
api.sound.calculateDistanceOptionsAtWorld(worldX: number, worldY: number, baseVolume?: number): SoundOptions;  
api.sound.stopById(soundId: string): void;  
api.sound.stopActive(): void;  
api.sound.stopAll(): void;
```

api.sprites

```
api.sprites.load(spriteId: string, path: string, options?: { tint?: number; }): Promise<void>;  
api.sprites.loadFromMod(spriteId: string, relativePath: string, options?: { tint?: number; }): Promise<void>;  
api.sprites.getById(spriteId: string): any;  
api.sprites.hideAllPlayerModSprites(): void;  
api.sprites.rotatePlayerModSprites(angle: number): void;
```

api.storage

```
api.storage.ensure(modId: string): any;  
api.storage.get(modId: string, key: string): any;  
api.storage.set(modId: string, key: string, value: any): void;  
api.storage.remove(modId: string, key: string): void;
```

api.storage.local

```
api.storage.local.get(key: string): any;  
api.storage.local.set(key: string, value: any): void;  
api.storage.local.remove(key: string): void;
```

api.settings

```
api.settings.get(fieldId: string): ConfigValueV1 | undefined;
api.settings.getAll(): Readonly<Record<string, ConfigValueV1>>;
api.settings.onChange(callback: (values: Readonly<Record<string, ConfigValueV1>>) => void): () => void;
```

api.structureBehaviors

```
api.structureBehaviors.registerConveyorType(structureId: string, options?: { transportOffset?: { x: number; y: number; }; velocity?: { x: number; y: number; }; maxTransportDistance?: number; transportHeight?: number; runWith?: 'left' | 'right'; skipQueued?: boolean; }): void;
api.structureBehaviors.registerLauncherType(definition: { upType: string; leftType: string; rightType: string; velocity: [number, number]; softDropVelocity: number; runTickSharedBufferKey?: string; }): void;
```

api.structures.recipes

```
api.structures.recipes.register(id: 'planterBox', definition: PlanterBoxRecipeDefinitionV1): void;
api.structures.recipes.register(id: 'shaker', definition: ShakerRecipeDefinitionV1): void;
api.structures.recipes.register(id: 'kineticPress', definition: KineticPressRecipeDefinitionV1): void;
api.structures.recipes.register(id: 'condenser' | 'steamDryer' | 'synthesizer' | 'snowmaker' | 'smelter', definition: WeightedRefineryRecipeDefinitionV1): void;
```

api.structures.processing

```
api.structures.processing.register(id: string, definition: StructureProcessingDefinitionV1): void;
api.structures.processing.isEnabledAt(cellX: number, cellY: number): boolean;
api.structures.processing.setEnabledAt(cellX: number, cellY: number, enabled: boolean): boolean;
```

api.structures

```
api.structures.addProcessor(structureId: string | StructureType, definition: StructureProcessorDefinitionV1): void;
api.structures.register(definition: SandkitStructureDefinition, options?: { useRawShape?: boolean; }): void;
api.structures.updateDefinition(structureTypeOrId: string | StructureType, partial: Partial<SandkitStructureDefinition>, options?: { useRawShape?: boolean; }): void;
api.structures.addVariant(baseStructureTypeOrId: string | StructureType, variant: { id: string | StructureType; angles: number[]; }, options?: { addBuildMode?: any; }): void;
api.structures.forEachOfType(structureTypeOrId: string | StructureType, callback: (structure: Structure) => void): void;
api.structures.registerPlacementConfig(definition: PlacementConfigDefinition): void;
api.structures.getAtCell(cellX: number, cellY: number): Structure | null;
api.structures.getDefinitionByType(structureType: string | StructureType): any;
api.structures.getUnlockedTypes(): Set<string | StructureType>;
api.structures.getTypeFromId(structureId: string): string | StructureType;
api.structures.hasBuiltAtCell(cellX: number, cellY: number): boolean;
```

```

api.structures.isBlockedByPlayerAtCell(cellX: number, cellY: number): boolean;
api.structures.isLauncherAtCell(cellX: number, cellY: number): boolean;
api.structures.isType(structure: Structure | null, structureId: string): boolean;
api.structures.isTypeAtCell(cellX: number, cellY: number, structureId: string): boolean;
api.structures.isUnlockedByType(structureType: string | StructureType): boolean;
api.structures.mapValueToSpritesheetIndex(value: number, thresholds: number[]): number;
api.structures.setSpritesheetIndex(structure: Structure, index: number): void;
api.structures.setSpritesheetIndexAtCell(cellX: number, cellY: number, index: number): void;
api.structures.setSpritesheetIndexByValue(structure: Structure, value: number, thresholds: number[]): void;
api.structures.setSpritesheetIndexByValueAtCell(cellX: number, cellY: number, value: number, thresholds:
number[]): void;
api.structures.update(structure: Structure, options?: { propagateToWorkers?: boolean; }): void;
api.structures.setData(structure: Structure, partial: any, options?: { propagateToWorkers?: boolean; }):
void;
api.structures.buildAtCellWhenIdle(cellX: number, cellY: number, structureTypeOrId: string, options?: any):
void;
api.structures.removeAtCellWhenIdle(cellX: number, cellY: number, options?: { removeCells?: boolean;
skipVisuals?: boolean; }): void;
api.structures.removeBetweenCellsWhenIdle(startCellX: number, startCellY: number, endCellX: number,
endCellY: number, options?: { removeCells?: boolean; preserveUnselectable?: boolean; onlyPositions?: { x:
number; y: number; }[]; }): void;
api.structures.removeAtCellsWhenIdle(positions: { x: number; y: number; }[], options?: { removeCells?:
boolean; skipVisuals?: boolean; }): void;

```

api.tech

```

api.tech.getDefinitionById(techId: string): any;
api.tech.updateDefinition(techId: string, updates: any): void;
api.tech.addDefinition(techId: string, definition: any): void;
api.tech.registerNode(techId: TechGridId, definition: TechDefinition, options: { parentId: TechGridId;
preferredPosition?: TechGridPosition; }): TechGridPosition;
api.tech.isLockedById(techId: string | number): boolean;
api.tech.setLockedById(techId: string | number, locked: boolean): void;

```

api.time

```

api.time.getTimeMs(): number;
api.time.getTick(): number;

```

api.tools.grabber

```

api.tools.grabber.setSize(size: number): void;
api.tools.grabber.getSize(): number;
api.tools.grabber.isActive(): boolean;
api.tools.grabber.isLoaded(): boolean;

```

api.terrains

```

api.terrains.register(definition: TerrainDefinition): { cellType: number; };
api.terrains.updateDefinition(cellTypeOrId: string | number, partial: Partial<TerrainDefinition>): void;
api.terrains.getTypeFromId(terrainId: string): number;
api.terrains.getTypeAtCell(cellX: number, cellY: number): number | null;
api.terrains.getDataAtCell(cellX: number, cellY: number): { cellType: number; hp: number | null; } | null;
api.terrains.isAtCell(cellX: number, cellY: number): boolean;
api.terrains.isTypeAtCell(cellX: number, cellY: number, terrainId: string): boolean;
api.terrains.isCellIdTerrain(cellId: number): boolean;
api.terrains.createAtCellWhenIdle(cellX: number, cellY: number, terrainTypeOrId: string | number, options?: TerrainMutationOptions): void;
api.terrains.replaceAtCellWhenIdle(cellX: number, cellY: number, terrainTypeOrId: string | number, options?: TerrainMutationOptions): void;
api.terrains.removeAtCellWhenIdle(cellX: number, cellY: number, options?: TerrainMutationOptions): void;
api.terrains.damageAtCell(cellX: number, cellY: number, damage: number): void;
api.terrains.setHpAtCellWhenIdle(cellX: number, cellY: number, hp: number): void;

```

api.triggers

```

api.triggers.register(triggerId: string, definition: MainTriggerDefinition): void;

```

api.ui

```

api.ui.update(componentId: ComponentId, options?: any): void;
api.ui.openPauseMenu(): void;
api.ui.showTooltip(data: TooltipData): void;
api.ui.toast(message: LocalizedText, options?: ToastOptions): void;
api.ui.alert(message: LocalizedText, title?: LocalizedText): Promise<void>;
api.ui.confirm(message: LocalizedText, title?: LocalizedText): Promise<boolean>;
api.ui.prompt(message: LocalizedText, defaultValue?: string, placeholder?: LocalizedText, title?: LocalizedText, allowCopy?: boolean): Promise<string | null>;
api.ui.inject(componentId: string, component: ComponentType<Record<string, never>>): () => void;

```

api.ui.overlays

```

api.ui.overlays.register(slot: string, overlayId: string, render: () => any): void;
api.ui.overlays.unregister(slot: string, overlayId: string): void;
api.ui.overlays.update(slot: string): void;

```

api.ui.navigation

```

api.ui.navigation.useFocusable<T extends HTMLElement = HTMLDivElement>(options: { readonly id: string; readonly scope: string; readonly onActivate: (element?: HTMLElement) => void; readonly onFocus?: (() => void); readonly disabled?: boolean; readonly x?: number; readonly y?: number; readonly neighbors?: Partial<Record<'left' | 'right' | 'up' | 'down', string>>; readonly scrollIntoView?: boolean; }): { readonly ref: RefObject<T>; readonly focused: boolean; readonly focus: () => void; };
api.ui.navigation.useFocusScope(options: { readonly id: string; readonly active: boolean; readonly priority?: number; readonly defaultId?: string; readonly onBack?: (() => boolean | void); }): void;

```

```
api.ui.navigation.controllerFocusClass(focused: boolean): string;
```

api.upgrades

```
api.upgrades.registerCategory(definition: UpgradeCategoryDefinition): void;  
api.upgrades.register(definition: UpgradeDefinition): void;  
api.upgrades.updateDefinition(itemId: string, upgradeId: string, partial: Record<string, any>): void;  
api.upgrades.getLevelById(itemId: string, upgradeId: string): number;  
api.upgrades.getAvailableLevelById(itemId: string, upgradeId: string): number;
```

api.utils

```
api.utils.getDistance(pointA: { x: number; y: number; }, pointB: { x: number; y: number; }): number;  
api.utils.getDirection(pointA: { x: number; y: number; }, pointB: { x: number; y: number; }): { x: number;  
y: number; };  
api.utils.getAngle(pointA: { x: number; y: number; }, pointB: { x: number; y: number; }): number;  
api.utils.getCoordinatesBetweenPoints(pointA: { x: number; y: number; }, pointB: { x: number; y: number; }):  
{ x: number; y: number; }[];
```

api.world

```
api.world.getCellIdAtCell(cellX: number, cellY: number): number;  
api.world.isCellEmptyAtCell(cellX: number, cellY: number): boolean;  
api.world.isTerrainAtCell(cellX: number, cellY: number): boolean;  
api.world.runWhenSimulationIdle(callback: () => void): void;  
api.world.reportActivityAtCell(cellX: number, cellY: number): void;  
api.world.excavateAtCell(cellX: number, cellY: number, outVelocity: { x: number; y: number; }, damage:  
number, options?: ExcavateOptions): void;  
api.world.revealFogAtCell(cellX: number, cellY: number): void;  
api.world.redrawAroundCellWhenIdle(cellX: number, cellY: number, range: number): void;
```

api.world.pickups

```
api.world.pickups.spawnAtWorld(type: WorldItemType, worldX: number, worldY: number, data?: any, light?:  
WorldItemLight): any;  
api.world.pickups.destroy(worldItem: any): void;  
api.world.pickups.pickUp(worldItem: any): boolean;  
api.world.pickups.getAll(): any[];  
api.world.pickups.getById(worldItemId: number): any;
```

api.shared.buffers

```
api.shared.buffers.create(key: string, config: { type: SharedArrayType; length: number; }): SharedArray;
api.shared.buffers.get(key: string): SharedArray | undefined;
```

api.workers

```
api.workers.setPostUpdateEnabled(enabled: boolean): void;
```

Worker entry

api.collector

```
api.collector.getValueFromCellId(cellId: number): number;
api.collector.getValueByType(elementType: number): number;
api.collector.isCellIdCollectable(cellId: number): boolean;
api.collector.isCellIdCollectableForSprite(cellId: number): boolean;
api.collector.notifyPickupAtCell(cellX: number, cellY: number): void;
```

api.effects

```
api.effects.createEffectAtWorld(effectId: string, worldX: number, worldY: number, options?: any): void;
api.effects.createLightAtWorld(worldX: number, worldY: number, options?: TemporaryLightOptions): { index:
number | null; };
api.effects.createParticlesAtWorld(worldX: number, worldY: number, options?: ParticleEffectOptions): void;
```

api.elements

```
api.elements.getTypeFromId(elementId: string): ElementType;
api.elements.getDefinitionByType(elementType: ElementType): ElementDefinition | undefined;
api.elements.getTypeAtCell(cellX: number, cellY: number): ElementType | null;
api.elements.getResolvedTypeAtCell(cellX: number, cellY: number): ElementType | null;
api.elements.getResolvedTypeFromCellId(cellId: number): ElementType | null;
api.elements.getInfoAtCell(cellX: number, cellY: number): { elementType: ElementType; isParticle: boolean;
cellId: number; elementIndex: number; } | null;
api.elements.getMatterTypeAtCell(cellX: number, cellY: number): MatterType | null;
api.elements.isTypeAtCell(cellX: number, cellY: number, elementType: ElementType): boolean;
api.elements.isFreeFallingAtCell(cellX: number, cellY: number): boolean;
api.elements.createAtCell(cellX: number, cellY: number, elementType: ElementType, options?:
ElementCreateOptions): void;
api.elements.replaceAtCell(cellX: number, cellY: number, elementType: ElementType, options?:
ElementCreateOptions): void;
api.elements.removeAtCell(cellX: number, cellY: number, options?: ElementRemovalOptions): void;
api.elements.moveBetweenCells(fromCellX: number, fromCellY: number, toCellX: number, toCellY: number):
```

```

boolean;
api.elements.teleportBetweenCells(fromCellX: number, fromCellY: number, toCellX: number, toCellY: number):
void;
api.elements.swapCells(firstCellX: number, firstCellY: number, secondCellX: number, secondCellY: number):
boolean;
api.elements.getVelocityAtCell(cellX: number, cellY: number): { x: number; y: number; } | null;
api.elements.setVelocityAtCell(cellX: number, cellY: number, velocity: { x: number; y: number; }): boolean;
api.elements.addParticleVelocityAtCell(cellX: number, cellY: number, velocity: { x: number; y: number; },
maxSpeed?: number): boolean;
api.elements.convertToParticleAtCell(cellX: number, cellY: number, velocity: { x: number; y: number; }):
boolean;
api.elements.convertFromParticleAtCell(cellX: number, cellY: number): boolean;
api.elements.getDataFieldAtCell(cellX: number, cellY: number, fieldNumber: 1 | 2 | 3 | 4): number | null;
api.elements.setDataFieldAtCell(cellX: number, cellY: number, fieldNumber: 1 | 2 | 3 | 4, value: number):
boolean;
api.elements.refreshColorAtCell(cellX: number, cellY: number): void;
api.elements.markMovementBlockedByElementIndex(elementIndex: number): void;
api.elements.setPhysicsAtCell(cellX: number, cellY: number, physicsState: number): void;
api.elements.setDurationAtCell(cellX: number, cellY: number, duration: number, options?: { updateMax?:
boolean; }): boolean;

```

api.events

```

api.events.on<Payload = any>(eventId: string, callback: (payload: Payload) => void, options?:
WorkerEventOptionsV1): () => void;
api.events.emit<Payload = any>(eventId: string, payload: Payload, options?: WorkerEventOptionsV1): void;

```

api.hooks

```

api.hooks.intercept<Args = any>(hookId: string, callback: (args: Args, context: HookContext) => void,
options?: WorkerHandlerOptionsV1): () => void;
api.hooks.modify<Args = any>(hookId: string, callback: (args: Args) => void, options?:
WorkerHandlerOptionsV1): () => void;

```

api.maps

```

api.maps.getActive(): Readonly<ActiveMapV1> | null;

```

api.fire

```

api.fire.canBurnElementAtCell(cellX: number, cellY: number): boolean;
api.fire.burnElementAtCell(cellX: number, cellY: number): boolean;

```

api.patterns

```
api.patterns.createCircle(size: number): number[][];
api.patterns.excavateAtCell(cellX: number, cellY: number, pattern: number[][], outVelocity: { x: number; y: number; }, power: number, options?: PatternExcavateOptions): void;
```

api.player

```
api.player.getWorldPosition(): { x: number; y: number; };
api.player.isCollidingWithCell(cellX: number, cellY: number): boolean;
api.player.isWithinRadiusOfCell(cellX: number, cellY: number, radius: number): boolean;
```

api.random

```
api.random.int(min: number, max: number): number;
api.random.float(min: number, max: number): number;
```

api.structures.processing

```
api.structures.processing.isEnabledAt(cellX: number, cellY: number): boolean;
```

api.structures

```
api.structures.getAtCell(cellX: number, cellY: number): Structure | null;
api.structures.getDefinitionByType(structureType: string | StructureType): any;
api.structures.getTypeFromId(structureId: string): string | StructureType;
api.structures.hasBuiltAtCell(cellX: number, cellY: number): boolean;
api.structures.isType(structure: Structure | null, structureId: string): boolean;
api.structures.isTypeAtCell(cellX: number, cellY: number, structureId: string): boolean;
api.structures.forEachOfType(structureTypeOrId: string | StructureType, callback: (structure: Structure) => void): void;
api.structures.update(structure: Structure, options?: { propagateToWorkers?: boolean; }): void;
api.structures.setData(structure: Structure, partial: any, options?: { propagateToWorkers?: boolean; }): void;
api.structures.setSpritesheetIndex(structure: Structure, index: number): void;
api.structures.setSpritesheetIndexAtCell(cellX: number, cellY: number, index: number): void;
api.structures.setSpritesheetIndexByValue(structure: Structure, value: number, thresholds: number[]): void;
api.structures.setSpritesheetIndexByValueAtCell(cellX: number, cellY: number, value: number, thresholds: number[]): void;
```

api.terrains

```
api.terrains.getTypeFromId(terrainId: string): number;
api.terrains.getTypeAtCell(cellX: number, cellY: number): number | null;
```

```
api.terrains.getDataAtCell(cellX: number, cellY: number): { cellType: number; hp: number | null; } | null;  
api.terrains.isAtCell(cellX: number, cellY: number): boolean;  
api.terrains.isTypeAtCell(cellX: number, cellY: number, terrainId: string): boolean;  
api.terrains.isCellIdTerrain(cellId: number): boolean;  
api.terrains.createAtCell(cellX: number, cellY: number, terrainTypeOrId: string | number, options?:  
TerrainMutationOptions): void;  
api.terrains.replaceAtCell(cellX: number, cellY: number, terrainTypeOrId: string | number, options?:  
TerrainMutationOptions): void;  
api.terrains.removeAtCell(cellX: number, cellY: number, options?: TerrainMutationOptions): void;  
api.terrains.damageAtCell(cellX: number, cellY: number, damage: number): void;  
api.terrains.setHpAtCell(cellX: number, cellY: number, hp: number): boolean;
```

api.ui

```
api.ui.toast(message: LocalizedText, options?: ToastOptions): void;
```

api.utils

```
api.utils.getDistance(pointA: { x: number; y: number; }, pointB: { x: number; y: number; }): number;  
api.utils.getDirection(pointA: { x: number; y: number; }, pointB: { x: number; y: number; }): { x: number;  
y: number; };  
api.utils.getAngle(pointA: { x: number; y: number; }, pointB: { x: number; y: number; }): number;  
api.utils.getCoordinatesBetweenPoints(pointA: { x: number; y: number; }, pointB: { x: number; y: number; }):  
{ x: number; y: number; }[];
```

api.world

```
api.world.getCellIdAtCell(cellX: number, cellY: number): number;  
api.world.isCellEmptyAtCell(cellX: number, cellY: number): boolean;  
api.world.isTerrainAtCell(cellX: number, cellY: number): boolean;  
api.world.reportActivityAtCell(cellX: number, cellY: number): void;  
api.world.excavateAtCell(cellX: number, cellY: number, outVelocity: { x: number; y: number; }, damage:  
number, options?: ExcavateOptions): void;
```

api.main

```
api.main.emitEvent<Payload = any>(eventId: string, payload: Payload): void;
```

api.worker

```
api.worker.getIndex(): number;  
api.worker.getCount(): number;
```

api.shared.buffers

```
api.shared.buffers.get(key: string): SharedArray | undefined;  
api.shared.buffers.require(key: string, config: { type: SharedArrayType; length: number; }): SharedArray;
```

Retrieved from "https://wiki.hoodedhorse.com/Sandustry/index.php?title=API_Reference&oldid=1080"